Name: Anushka Suvarna
UID: 2023300241
Batch: B
Course Code: CS303
Experiment no.: 3
Title of the Experiment: To implement AND Gate using McCulloch Pitts Neuron and Design Half and Full Adder

Theory:

This experiment focuses on understanding how logical operations and arithmetic units can be implemented using the McCulloch-Pitts (MCP) neuron model, which is the earliest mathematical model of a biological neuron. The MCP neuron works by summing weighted binary inputs and applying a threshold function to determine the output.

I first implemented the AND gate using a single MCP neuron by setting appropriate weights and a threshold. This demonstrates how basic logic gates can be modeled with simple neural computation.

In the second part of the experiment, I extended the design to arithmetic circuits. Using MCP neurons to construct XOR gates, I implemented a Half Adder, which performs binary addition of two bits and produces a Sum and a Carry. I also designed a Full Adder using MCP neurons, which takes three inputs (two binary bits and a carry-in) and produces the correct Sum and Carry outputs.

This shows how multiple MCP neurons can be combined to model more complex digital systems.

Procedure:

Part 1: AND Gate using McCulloch Pitts Neuron

1. Theory:

The McCulloch-Pitts (MCP) neuron is one of the earliest models of an artificial neuron, introduced to explain how logical operations can be represented mathematically. It takes a set of binary inputs, applies weights, computes their sum, and then compares the result with a threshold. If the sum is greater than or equal to the threshold, the neuron outputs 1; otherwise, it outputs 0.

The AND gate is a fundamental logic gate in digital systems. It produces an output of 1 only when both inputs are 1, otherwise the output is 0. To implement this using an MCP neuron:

- Each input is assigned a weight.
- The threshold is set such that the output is activated (1) only when both inputs are 1.

For example:

- Inputs: x1, x2
- Weights: w1=1, w2=1
- Threshold: θ=2

This configuration ensures that the neuron outputs 1 only when both inputs are 1, thus mimicking the behavior of an AND gate.

2.Design Steps:

1. Represent the inputs as binary values x1, x2 ∈ {0,1}
2. Assign weights to the inputs: w1=1, w2=1
3. Define the threshold θ=2
4. Compute the weighted sum: net = x1 · w1+ x2 · w2
5. Apply the activation function (step function):
    - If net ≥ θ, output = 1.
    - Else, output = 0.
6. Verify the output for all four possible input combinations to complete the truth table of the AND gate.

3.Program Code:

```
def mcp_and(x1,x2):

    w1,w2=1,1

    theta=2

    summation=x1*w1+x2*w2

    if summation>=theta:

        return 1

    else:

        return 0

print("AND Gate using MCP Neuron")

print("0 AND 0 =", mcp_and(0,0))

print("0 AND 1 =", mcp_and(0,1))
```

```
print("1 AND 0 =", mcp_and(1,0))

print("1 AND 1 =", mcp_and(1,1))
```

Output:

```
C:\Users\SUVARNA HOUSE\Downloads\AISC>python mcculloch_and.py
AND Gate using MCP Neuron
0 AND 0 = 0
0 AND 1 = 0
1 AND 0 = 0
1 AND 1 = 1

C:\Users\SUVARNA HOUSE\Downloads\AISC>
```

Part 2: Half Adder and Full Adder using McCulloch Pitts Neuron

1. Theory:

Half Adder:
A Half Adder is a basic combinational circuit in digital electronics that adds two binary digits and produces two outputs: Sum and Carry. It is widely used as the building block for more complex arithmetic circuits.

In terms of logic:

- Sum is obtained using the XOR operation.
- Carry is obtained using the AND operation.

When implemented with McCulloch-Pitts (MCP) neurons:

- Each logic gate (AND, OR, NOT, XOR) can be modeled using MCP neurons with suitable weights and thresholds.
- The Sum (XOR) is realized by combining MCP neurons that perform OR, AND, and NOT operations.
- The Carry (AND) is realized directly with an MCP neuron where both inputs must be 1 to activate the output.

Thus, the Half Adder demonstrates how MCP neurons can be interconnected to perform arithmetic operations, proving that neural models can implement not just logic gates but also fundamental digital circuits.

Full Adder:

A Full Adder is a fundamental combinational circuit in digital electronics that adds three binary inputs—two significant bits (AAA and BBB) and a carry-in (CinC_{in}Cin)—and produces two outputs: Sum and Carry. It is an essential building block for constructing multi-bit binary adders.

In terms of logic:

- Sum is obtained using the expression:

  $$Sum = A \oplus B \oplus C_{in}$$

- Carry is obtained using the expression:

  $$Carry = (A \cdot B) + (Cin \cdot (A \oplus B))$$

When implemented with McCulloch-Pitts (MCP) neurons:

- Each logic gate (AND, OR, NOT, XOR) is modeled using MCP neurons with appropriate weights and thresholds.
- The Sum (XOR of three inputs) is constructed by cascading MCP-based XOR neurons.
- The Carry is realized by combining MCP-based AND neurons (for $A \cdot B$ and $Cin \cdot (A \oplus B)$) and then feeding them into an MCP-based OR neuron.

Thus, the Full Adder illustrates how multiple MCP neurons can be interconnected to perform more complex arithmetic operations. It demonstrates that even advanced digital circuits can be systematically modeled using the simple principles of the McCulloch-Pitts neuron.

2. Design Steps:

Half Adder:

1) Represent the two binary inputs as A and B.
2) Implement the AND gate using an MCP neuron to generate the Carry output.
3) Implement the XOR gate using a combination of MCP neurons (OR, AND, and NOT) to generate the Sum output.
4) Connect the outputs of the MCP neurons such that:
   - $Sum = A \oplus B$
   - $Carry = A \cdot B$
5) Prepare the truth table for all four input combinations (0,0), (0,1), (1,0), (1,1)

6) Verify that the Sum and Carry outputs match the expected Half Adder truth table.

Full Adder:

1. Represent the three binary inputs as A, B, and C_{in} (carry-in).
2. Implement the XOR gate using MCP neurons (OR, AND, and NOT) to generate the partial sums.
3. Compute the first XOR: $X1=B \oplus C_{in}$.
4. Compute the final Sum using another XOR: $Sum= A \oplus X_1$
5. Generate the carry outputs using MCP-based AND and OR gates:

    - $Carry_1=A \cdot B$
    - $Carry_2=C_{in} \cdot X_1$
    - Final Carry = $Carry_1 + Carry_2$ (implemented using OR neuron).

6. Connect the MCP neurons in sequence to realize both Sum and Carry outputs.
7. Prepare the truth table for all eight input combinations of A, B, C_{in}
8. Verify that the outputs match the Full Adder truth table.

3.Program Code

```
def mcp_and(x1,x2):
    w1,w2=1,1
    theta=2

    if (x1*w1+x2*w2)>=theta:
        return 1
    else:
        return 0

def mcp_or(x1,x2):
    w1,w2=1,1
    theta=1

    if (x1*w1+x2*w2)>=theta:
        return 1
```

```
    else:
        return 0

def mcp_not(x):
    w=-1
    theta=0

    if(x*w)>=theta:
        return 1
    else:
        return 0

def mcp_xor(x1,x2):
    term1=mcp_or(x1,x2)
    term2=mcp_and(x1,x2)
    term2_not=mcp_not(term2)
    return mcp_and(term1,term2_not)

def half_adder(a,b):
    s=mcp_xor(a,b)
    carry=mcp_and(a,b)
    return s,carry

def full_adder(a,b,cin):

    xor1=mcp_xor(b,cin)
    s=mcp_xor(a,xor1)
```

```
    carry1=mcp_and(a,b)
    carry2=mcp_and(cin,xor1)
    carry=mcp_or(carry1,carry2)

    return s,carry

print("Half Adder:")
for a in [0,1]:
    for b in [0,1]:
        s,c=half_adder(a,b)
        print(f"A={a}, B={b} : Sum={s}, Carry={c}")

print("\nFull Adder:")
for a in [0,1]:
    for b in [0,1]:
        for cin in [0,1]:
            s,c=full_adder(a,b,cin)
            print(f"A={a}, B={b}, Cin={cin} : Sum={s}, Carry={c}")
```

Output:

```
C:\Users\SUVARNA HOUSE\Downloads\AISC>python mcculloch_adders.py
Half Adder:
A=0, B=0 : Sum=0, Carry=0
A=0, B=1 : Sum=1, Carry=0
A=1, B=0 : Sum=1, Carry=0
A=1, B=1 : Sum=0, Carry=1

Full Adder:
A=0, B=0, Cin=0 : Sum=0, Carry=0
A=0, B=0, Cin=1 : Sum=1, Carry=1
A=0, B=1, Cin=0 : Sum=1, Carry=0
A=0, B=1, Cin=1 : Sum=0, Carry=0
A=1, B=0, Cin=0 : Sum=1, Carry=0
A=1, B=0, Cin=1 : Sum=0, Carry=1
A=1, B=1, Cin=0 : Sum=0, Carry=1
A=1, B=1, Cin=1 : Sum=1, Carry=1

C:\Users\SUVARNA HOUSE\Downloads\AISC>
```

Conclusion:

In this experiment, I implemented basic logic gates using McCulloch-Pitts neurons and then designed Half Adder and Full Adder circuits. I learned how simple neurons with fixed weights and thresholds can be combined to perform arithmetic operations. This showed me that neural models can represent not only logic gates but also more complex digital circuits. Overall, the experiment helped me understand the link between neural computation and digital electronics.